LAPORAN TUGAS KECIL

IF2211 Strategi Algoritma

Semester II tahun 2025/2026

***Voxelization* Objek 3D menggunakan Octree**

Dipersiapkan oleh:

Benedict Darrel Setiawan – 13524057

Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung

Jl. Ganesha 10, Bandung 40132

1. Pendahuluan

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

Fungsi dari program yang dirancang adalah untuk mengubah sebuah model biasa dalam bentuk file .obj ke dalam sebuah model yang murni terdiri dari voxel yang memiliki rusuk yang sama serta menkonversikannya ke dalam bentuk file .obj. Hal ini akan dilakukan dengan memanfaatkan algoritma Divide and Conquer yang merupakan sebuah algoritma yang membagi sebuah persoalan ke bagian-bagian kecil dan menggabungkan solusi agar membentuk solusi persoalan yang awal

2. Penjelasan Algoritma

Langkah-langkah algoritma Divide and Conquer yang dirancang untuk membangun program ini adalah sebagai berikut

1. Dapatkan seluruh vertex dan face dari model .obj awal
2. Transformasikan seluruh vertex agar memiliki titik tengah *c* dan panjang rusuk bounding box maksimum *r*
3. Definisikan sebuah voxel/kubus dengan suatu vertex tengah *c* dan panjang rusuk *r*. Simpan juga sebuah list yang mengandung seluruh face dari objek awal
4. Bagi voxel/kubus tersebut menjadi 8 voxel/kubus. Child voxel memiliki vertex tengah yang merupakan pertengahan antara vertex tengah dan satu vertex ujung parent voxel, serta memiliki panjang rusuk *r/2* dimana *r* merupakan panjang rusuk parent voxel.
5. Untuk setiap child voxel carilah seluruh face dari list face yang dimiliki parent voxel yang dianggap 'menyentuh' dirinya. Sebuah face dianggap menyentuh voxel jika :
    a. Mengandung sebuah vertex yang berada di daerah voxel
    b. Berpotongan dengan voxel secara langsung. Hal ini diperiksa dengan cara memproyeksikan voxel dan face ke *(x,y), (x,z), (y,z)*. Jika di ketiga proyeksi, face dan kubus berpotongan atau berada di dalam yang lain, maka voxel dan face dianggap berpotongan

    Seluruh face yang dianggap menyentuh voxel akan disimpan ke dalam sebuah list
6. Jika panjang dari list face yang bersentuhan dengan voxel adalah 0, maka voxel dianggap sebagai dead node
7. Ulangi Langkah 4 – 6 hingga mencapai kedalaman *n*

Kode untuk algoritma Divide and Conquer yang digunakan di dalam program ini dalam bahasa Go adalah sebagai berikut

```
type Octree struct {
   value          bool
   object         *object.Object
   voxel          Voxel
   intersectingFaces []int
   children       [8]*Octree
}

func NewOctreeRoot(O *object.Object) *Octree {
   face_idx_list := []int{}
   for i := range O.FaceArray {
      face_idx_list = append(face_idx_list, i)
   }
   voxel := Voxel{*object.NewVertex(0, 0, 0), MaxEdgeLength}
   return &Octree{true,
      O,
      voxel,
      face_idx_list,
      [8]*Octree{nil, nil, nil, nil, nil, nil, nil, nil}}
}

func NewOctree(voxel Voxel, fl []int, O *object.Object) *Octree {
   return &Octree{true,
      O,
      voxel,
      fl,
      [8]*Octree{nil, nil, nil, nil, nil, nil, nil, nil}}
}


func IncreaseDepth(Oct *Octree) {
   if isLeaf(Oct) && Oct.value {
      voxelCorners := GetVoxelVertices(&Oct.voxel)
      for i := range Oct.children {
         subVox := Voxel{*object.MidVertex(&Oct.voxel.center, voxelCorners[i]), Oct.voxel.edgeLength / 2}
         new_i_f := GetIntersectingFaces(vox, Oct.intersectingFaces)
         Oct.children[i] = NewOctree(subVox, new_i_f, Oct.object)
         Oct.children[i].value = (len(new_i_f) > 0)
      }
   } else if Oct.value {
      for _, p := range Oct.children {
         IncreaseDepth(p)
      }
   }
}
```

Penjelasan atas setiap fungsi dan prosedur yang berdampak adalah sebagai berikut

a. *GetVoxelVertices*(vox Voxel)
   Fungsi yang mengembalikan 8 vertex yang membentuk kubus.
b. *MidVertex*(v1, v2 Vertex)
   Fungsi yang vertex yang berada ditengah v1 dan v2. Memiliki kompleksitas waktu $O(1)$
c. *isPolygonClippingVoxel*(vox Voxel, f_list []int)
   Fungsi yang mengembalikan boolean. True jika salah satu dari kedua kondisi tersebut benar:

i. Mengandung sebuah vertex yang berada di daerah voxel
ii. Berpotongan dengan voxel secara langsung. Hal ini diperiksa dengan cara memproyeksikan voxel dan face ke *(x,y), (x,z), (y,z)*. Jika di ketiga proyeksi, face dan kubus berpotongan atau berada di dalam yang lain, maka voxel dan face dianggap berpotongan

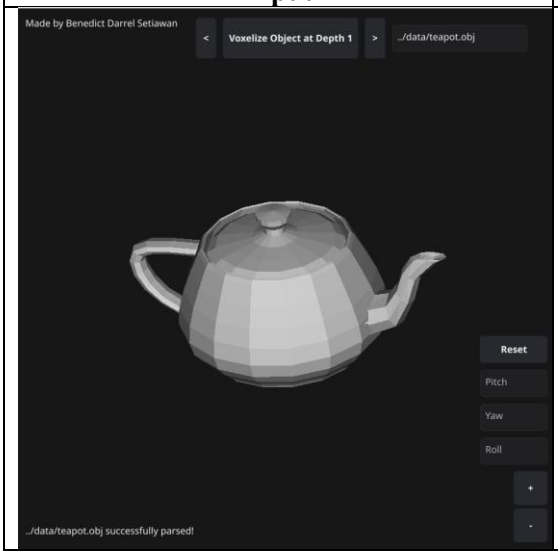
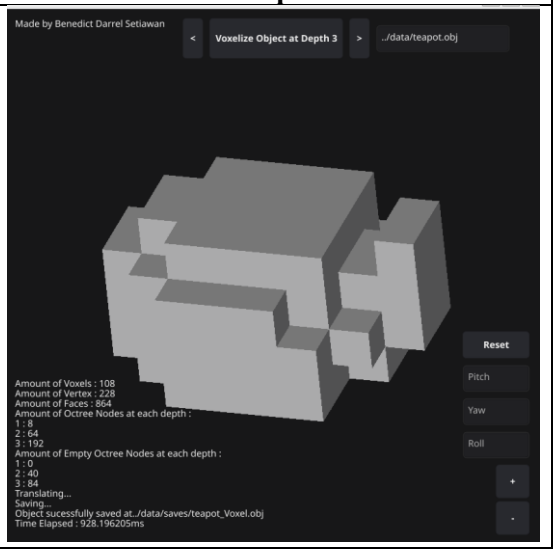
Karena setiap face hanya memiliki 3 edges tetap, setiap proyeksi voxel hanya memiliki ke-empat edge luar tetap dan proyeksi ada 3 macam yang tetap, maka kompleksitas waktu dari fungsi ini adalah *O(3*3*4*n) atau O(n)* dengan *n* merupakan jumlah dari faces di list

3. Perancangan Program

Program Voxelization ini dibangun menggunakan bahasa pemrograman Go. Hal ini dilakukan agar memudahkan melakukan concurrency dengan goroutines.

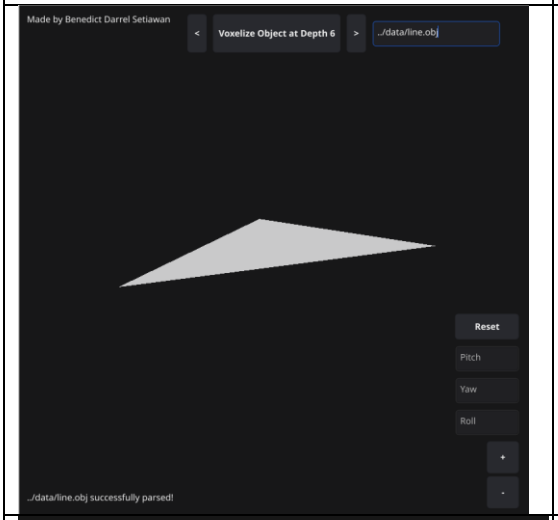
Program ini memiliki kemampuan untuk memvisualisasikan model .obj baik sebelum ataupun sesudah Voxelization. 3d model viewer yang dibuat mengandung fungsi rotation lewat drag maupun input melalui value, dan juga fitur zoom in dan out yang sederhana.

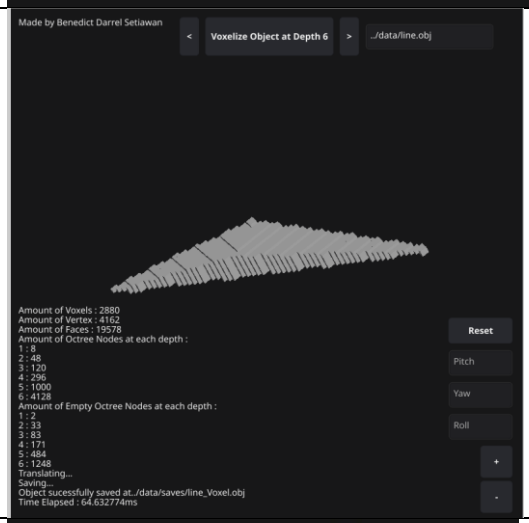
4. Pengujian Program

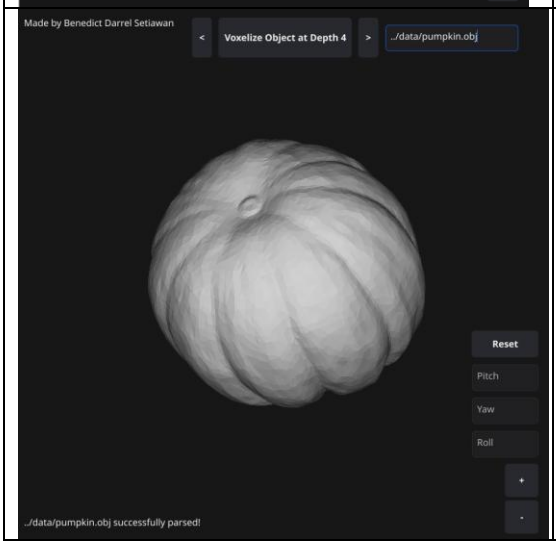
Input	Output
	

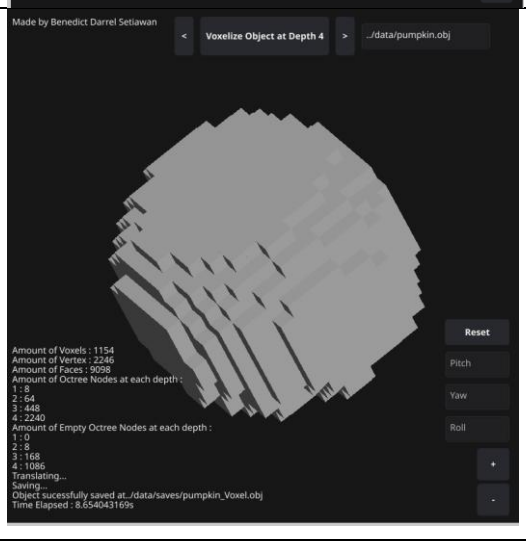
Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 3
>
../data/cow.obj
Reset
Pitch
Yaw
Roll
+
-
../data/cow.obj successfully parsed!

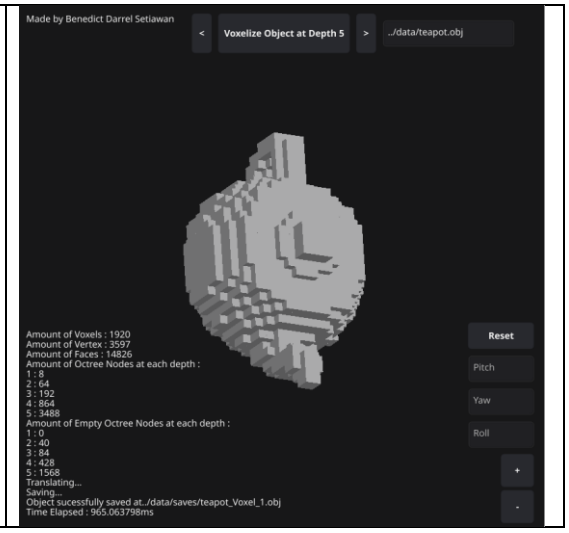
Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 3
>
../data/cow.obj
Reset
Pitch
Yaw
Roll
+
-
Amount of Voxels : 88
Amount of Vertex : 198
Amount of Faces : 700
Amount of Octree Nodes at each depth :
1 : 8
2 : 64
3 : 160
Amount of Empty Octree Nodes at each depth :
1 : 0
2 : 44
3 : 72
Translating...
Saving...
Object sucessfully saved at../data/saves/cow_Voxel.obj
Time Elapsed : 4.99093115s


Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 6
>
../data/line.obj
Reset
Pitch
Yaw
Roll
+
-
../data/line.obj successfully parsed!


Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 6
>
../data/line.obj
Reset
Pitch
Yaw
Roll
+
-
Amount of Voxels : 2880
Amount of Vertex : 4162
Amount of Faces : 19578
Amount of Octree Nodes at each depth :
1 : 8
2 : 48
3 : 120
4 : 296
5 : 1000
6 : 4128
Amount of Empty Octree Nodes at each depth :
1 : 2
2 : 33
3 : 83
4 : 171
5 : 484
6 : 1248
Translating...
Saving...
Object sucessfully saved at../data/saves/line_Voxel.obj
Time Elapsed : 64.632774ms


Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 4
>
../data/pumpkin.obj
Reset
Pitch
Yaw
Roll
+
-
../data/pumpkin.obj successfully parsed!


Made by Benedict Darrel Setiawan
<
Voxelize Object at Depth 4
>
../data/pumpkin.obj
Reset
Pitch
Yaw
Roll
+
-
Amount of Voxels : 1154
Amount of Vertex : 2246
Amount of Faces : 9098
Amount of Octree Nodes at each depth :
1 : 8
2 : 64
3 : 448
4 : 2240
Amount of Empty Octree Nodes at each depth :
1 : 0
2 : 8
3 : 168
4 : 1086
Translating...
Saving...
Object sucessfully saved at../data/saves/pumpkin_Voxel.obj
Time Elapsed : 8.654043169s



5. Lampiran

Repository proyek : https://github.com/BenedictD-RH/Tucil2_13524057

No	Poin	Ya	Tidak
1.	Program berhasil di kompilasi tanpa kesalahan	✓	
2.	Program berhasil di jalankan	✓	
3.	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4.	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	✓	
5.	Program memiliki Graphical User Interface (GUI)	✓	
6.	Program dapat menyimpan solusi dalam bentuk file gambar	✓	

6. Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Benedict Darrel Setiawan